5.10 Integration of System Components

CurriculumAI integrates two workflows through Django and PostgreSQL. In the first, an administrator submits a source and a background worker prepares its passages for retrieval. In the second, a student submits a question and Django uses the indexed passages to decide whether it can return a cited answer. The database links these workflows through source status, passage embeddings and citation records.

Browser, API and deployment

The React interface sends requests to Django REST Framework through a shared API client. The client includes the browser's session cookies and obtains a CSRF token before requests that change state. Django checks the requester's role and, for conversation endpoints, limits access to the conversation owner. Under Docker Compose, Vite proxies browser requests under **/api/** to Django. The API and ingestion worker connect to the same PostgreSQL service and share a private media volume, so the worker can open a document stored by the upload endpoint. Both services depend on database readiness and successful migrations.

The API client also translates failed requests into messages the interface can display. For a registered student retrying a failed turn, the chat interface checks the saved conversation first. A response may already have been saved even if the browser stopped waiting, so this check helps avoid an unnecessary second generation request.

From source submission to retrieval

A document upload sends multipart form data to the administrator's source endpoint. Django validates the file, stores it privately and creates a source record with status **QUEUED**. Website registration stores a validated URL for the worker to fetch during processing. The upload response confirms that the source has entered the queue. The administrator can then inspect its processing status through a separate API request.

The ingestion worker claims the queued source, extracts its text, and stores passages with their source locations. It obtains embeddings through the OpenAI adapter and writes them to PostgreSQL/pgvector. Once the worker verifies that the passages, metadata and vectors are complete, it sets the source to **INDEXED**. The retrieval query filters for eligible indexed passages, so accepting an upload does not expose incomplete material to students. A failed source remains excluded and the administrator can retry its ingestion.

From question submission to a displayed answer

The chat interface submits the question to **/api/qa/answer/**. A signed-in student's request includes the conversation identifier, which Django checks before loading the relevant history. The backend can also reuse a matching unanswered turn when a student retries. A guest request can include bounded conversation context held by the browser. Guest turns are kept out of the conversation database.

Django embeds the retrieval query with the model used for passage embeddings. PostgreSQL/pgvector ranks eligible passages by exact cosine distance and returns a bounded set of candidates. Django labels those passages and supplies them to the Tier 1 model with the original question and bounded history. The passages provide evidence, while the history helps the model interpret follow-up questions. The model assesses whether the evidence covers every substantive part of the question and proposes a cited response when it judges the support sufficient.

Django checks the response structure, citation markers and exact quotations, then revalidates the cited source evidence before acceptance. For registered students, it saves the encrypted assistant message and citation snapshots in one transaction. The response returned to React includes the tier and citation details used to render expandable references. On reopening a conversation, Django retrieves the owner's saved messages and citation snapshots, allowing the interface to show the source details associated with the original answer.

External fallback and error handling

If Tier 1 cannot produce an accepted answer for an eligible reason, Django sends the original question and bounded history to the configured Tier 2 model. The Tier 2 request uses fixed instructions for general Scala assistance and excludes the current retrieved passages and rejected Tier 1 answer. Django validates the returned text and supplies the external-answer label and reason notice. React displays these alongside the answer and omits course references. Registered Tier 2 responses retain the fallback reason in the saved conversation.

The backend controls the distinction between recoverable failures and errors. A stale source can qualify for fallback, whereas a database error during the final evidence check returns a controlled error. Generation-provider refusals or outages, access violations and persistence failures likewise remain controlled errors. This preserves the reason for the outcome when it reaches the interface.

Verification of the integrated workflows

The retained runs from 30 October 2026 record passes for the six cases below against the same source fingerprint. The backend tests used isolated PostgreSQL/pgvector, temporary storage and injected AI providers. The browser test simulated account API responses. The full-system tests used a disposable Docker environment, real browser requests and live OpenAI services.

Case	Test conditions	Observed result
I01	Django REST Framework APIClient, real pgvector, mocked embeddings	A synthetic PDF entered the queue. Direct invocation of the ingestion worker indexed it, and retrieval returned the extracted passage with its page location.
I02	Django REST Framework APIClient, real pgvector, mocked embeddings and generation	An uploaded PDF became retrievable evidence for a validated Tier 1 answer. The test checked two encrypted messages and one citation snapshot linked to the stored passage.
B08	Browser with simulated account API responses	Registration opened the chat interface without requesting the unavailable deferred chat bundle. The test checked navigation within the existing browser document.
S01	Browser, Django, PostgreSQL/pgvector and live OpenAI	A registered student received a cited Tier 1 answer. Reopening the conversation restored the saved answer and expandable citation details.
S02	Guest browser session and live OpenAI	A question outside the test corpus received a Tier 2 answer with the external label and a reason notice. The response contained no course citations.
S03	Administrator upload, independent worker and live OpenAI	The browser observed the source move from QUEUED to INDEXED. A later student answer cited that newly indexed source.

Together, these checks establish that the tested source, answer and history workflows connect across the implemented interfaces. The results concern system behaviour under the recorded conditions. Chapter 6 evaluates answer quality, citation faithfulness and student experience separately.

Execution records. The repository retains the integration run docker-thesis-20261030t105350z-c5552b, browser run docker-thesis-20261030t110412z-3b693d, and live system run docker-thesis-20261030t110454z-abee72 under test/reports/.